# Estimación de la Curva de Phillips para España

En esta práctica estudiamos la relación a medio plazo entre la inflación y la tasa de paro en la economía española mediante la estimación empírica de una **Curva de Phillips**. Para ello utilizamos datos trimestrales procedentes de la base de datos BDREMS:

- **Tasa de paro trimestral** de España.
- **Deflactor del gasto en consumo final de los Hogares + ISFSH**, también trimestral.

A partir del deflactor de consumo construiremos la **inflación anual** como tasa de crecimiento logarítmica a cuatro trimestres:

$$\pi_t = 100 \cdot [\ln(P_t) - \ln(P_{t-4})].$$

Supondremos además un mecanismo de **expectativas adaptativas**, de modo que la inflación esperada dentro de un año es la inflación observada hoy:

$$\pi^e_{t+4} = \pi_t.$$

Bajo esta hipótesis, la Curva de Phillips se expresa como:

$$\Delta_4\pi_t = \beta_0 + \beta_1 u_t + \varepsilon_t,$$

donde evaluamos:

- Variación anual de la inflación.
- Tasa de paro.

El objetivo principal es:

- **estimar la tasa natural de paro**
- **calcular su desviación típica** mediante el método Delta,
- y extender posteriormente el análisis para considerar **distintos regímenes de inflación**.

In [2]:
```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_style("whitegrid")
```

In [3]:
```
fuente = "datos_phillips.xlsx"
df = pd.read_excel(fuente)

# Comprobar columnas
print(df.columns)

# Construimos una fecha aproximada: primer mes de cada trimestre
# (T1 -> marzo, T2 -> junio, T3 -> septiembre, T4 -> diciembre)
mes = df['trimestre'] * 3
df['fecha'] = pd.to_datetime(
    dict(year=df['anio'], month=mes, day=1)
)

# Ordenar por fecha
df = df.sort_values(['anio', 'trimestre']).reset_index(drop=True)

df.head()
```

```
Index(['anio', 'trimestre', 'precio', 'tasa_paro', 'log ', 'pi_t',
       'delta4_pi'],
      dtype='object')
```

Out[3]:

| | anio | trimestre | precio | tasa_paro | log | pi_t | delta4_pi | fecha |
|---|---|---|---|---|---|---|---|---|
| **0** | 1980 | 1 | 0.177322 | 9.706395 | -1.729789 | NaN | NaN | 1980-03-01 |
| **1** | 1980 | 2 | 0.195594 | 10.425212 | -1.631714 | NaN | NaN | 1980-06-01 |
| **2** | 1980 | 3 | 0.210145 | 10.904520 | -1.559956 | NaN | NaN | 1980-09-01 |
| **3** | 1980 | 4 | 0.218290 | 11.540145 | -1.521930 | NaN | NaN | 1980-12-01 |
| **4** | 1981 | 1 | 0.198868 | 12.168170 | -1.615115 | 11.467486 | NaN | 1981-03-01 |

## Parte 1: Estimación de la tasa natural de paro (ū) con toda la muestra

En esta primera parte utilizaremos todos los datos disponibles en BDREMS para estimar una Curva de Phillips simple basada en expectativas adaptativas. La ecuación a estimar es:

$$\Delta_4\pi_t = \beta_0 + \beta_1 u_t + \varepsilon_t.$$

A partir de la teoría de la Curva de Phillips con expectativas, existe una relación entre los parámetros estimados y la **tasa natural de paro**:

$$\beta_0 = \alpha\bar{u}, \qquad \beta_1 = -\alpha.$$

Por tanto, el estimador es:

$$\hat{\bar{u}} = -\frac{\hat{\beta}_0}{\hat{\beta}_1}.$$

Para evaluar la precisión de esta estimación, calcularemos la **desviación típica** de (\hat{\bar{u}}) aplicando el **método Delta**, utilizando el gradiente de la función no lineal:

$$f(\beta_0, \beta_1) = -\frac{\beta_0}{\beta_1}.$$

y la matriz de varianzas-covarianzas.

Al finalizar esta parte habremos obtenido:

- La estimación puntual de la tasa de paro natural.
- Su desviación típica y, opcionalmente, un intervalo de confianza.

In [4]:
```
# Logaritmo del precio
df['log_P'] = np.log(df['precio'])

# Inflación anual ln(π_t) = ln(P_t) - ln(P_{t-4}*100)
df['pi_t'] = (df['log_P'] - df['log_P'].shift(4)) *100

# Δ4π_t = π_t - π_{t-4}
df['delta4_pi'] = df['pi_t'] - df['pi_t'].shift(4)

# Eliminamos observaciones iniciales sin datos completos
df_model = df.dropna(subset=['delta4_pi', 'tasa_paro']).copy()

df_model[['log_P','fecha', 'tasa_paro', 'pi_t', 'delta4_pi']].head()
```

Out[4]:

| | log_P | fecha | tasa_paro | pi_t | delta4_pi |
|---|---|---|---|---|---|
| **8** | -1.466049 | 1982-03-01 | 14.035329 | 14.906513 | 3.439027 |
| **9** | -1.363573 | 1982-06-01 | 14.309347 | 13.762080 | 0.710053 |
| **10** | -1.304598 | 1982-09-01 | 14.987015 | 12.044711 | -1.446437 |
| **11** | -1.262152 | 1982-12-01 | 15.318076 | 12.053207 | -1.871354 |
| **12** | -1.349363 | 1983-03-01 | 15.613193 | 11.668625 | -3.237888 |

In [5]:
```
Y = df_model['delta4_pi']            # Δ4π_t
X = df_model[['tasa_paro']]          # u_t
X = sm.add_constant(X)               # añade intercepto β0

model1 = sm.OLS(Y, X)
res1 = model1.fit()

print(res1.summary())
```

```
                            OLS Regression Results                            
==============================================================================
Dep. Variable:              delta4_pi   R-squared:                       0.038
Model:                            OLS   Adj. R-squared:                  0.032
Method:                 Least Squares   F-statistic:                     6.374
Date:                Sun, 23 Nov 2025   Prob (F-statistic):             0.0125
Time:                        22:22:06   Log-Likelihood:                -351.32
No. Observations:                 165   AIC:                             706.6
Df Residuals:                     163   BIC:                             712.8
Df Model:                           1                                         
Covariance Type:            nonrobust                                         
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          1.3224      0.611      2.163      0.032       0.115       2.530
tasa_paro     -0.0935      0.037     -2.525      0.013      -0.167      -0.020
==============================================================================
Omnibus:                        9.044   Durbin-Watson:                   0.700
Prob(Omnibus):                  0.011   Jarque-Bera (JB):               14.491
Skew:                           0.256   Prob(JB):                     0.000713
Kurtosis:                       4.358   Cond. No.                         63.5
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

In [6]:
```
b_hat = res1.params
b0_hat = b_hat['const']
b1_hat = b_hat['tasa_paro']

u_bar_hat = - b0_hat / b1_hat
print(f"(ū^) = {u_bar_hat:.4f}")
```

```
(ū^) = 14.1389
```

In [7]:
```
# Matriz de covarianzas de (β0, β1)
cov_beta1 = res1.cov_params().loc[['const', 'tasa_paro'], ['const', 'tasa_paro']]

# Gradiente de f(β0, β1) = -β0/β1
g1 = -1.0 / b1_hat
g2 = b0_hat / (b1_hat**2)
G = np.array([[g1],
              [g2]])  # 2x1

Sigma1 = cov_beta1.to_numpy()

var_u_bar_hat = float(G.T @ Sigma1 @ G)
std_u_bar_hat = np.sqrt(var_u_bar_hat)

print(f"Varianza(ū^) ≈ {var_u_bar_hat:.6f}")
print(f"Desviación típica(ū^) ≈ {std_u_bar_hat:.6f}")
```

```
Varianza(ū^) ≈ 3.407395
Desviación típica(ū^) ≈ 1.845913
/var/folders/tk/lzgzrvjs12n47rkb247_bbnm0000gn/T/ipykernel_6894/1893832733.py:12: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)
  var_u_bar_hat = float(G.T @ Sigma1 @ G)
```

In [8]:
```
z = 1.96
lower = u_bar_hat - z * std_u_bar_hat
upper = u_bar_hat + z * std_u_bar_hat

print(f"IC 95% (ū^): [{lower:.4f}, {upper:.4f}]")
```

```
IC 95% (ū^): [10.5209, 17.7569]
```

## Parte 2: Curva de Phillips con regímenes de inflación alta y baja

En esta segunda parte exploraremos si la relación entre inflación y paro cambia dependiendo del nivel de inflación observado. Para ello dividiremos la muestra en dos regímenes:

- **Inflación alta**: $\pi_t > 4\%$
- **Inflación baja**: $\pi_t \leq 4\%$

Estimaremos una Curva de Phillips con **parámetros dependientes del régimen**:

$$\Delta_4\pi_t = \beta_{01}\xi_t + \beta_{02}(1-\xi_t) + \beta_{11}\xi_t u_t + \beta_{12}(1-\xi_t)u_t + \varepsilon_t,$$

donde (\xi_t) es una variable indicadora que vale 1 en periodos de inflación alta y 0 en periodos de inflación baja.

Esto nos permite obtener dos tasas naturales de paro:

$$\bar{u}_1 = -\frac{\beta_{01}}{\beta_{11}} \qquad \text{(inflación alta)},$$

$$\bar{u}_2 = -\frac{\beta_{02}}{\beta_{12}} \qquad \text{(inflación baja)}.$$

Al igual que en la Parte 1, calcularemos la **desviación típica** de cada estimación mediante el método Delta, usando las submatrices de la matriz de covarianzas asociadas a cada par ((\beta_{0j},\beta_{1j})).

Finalmente evaluaremos si:

- La tasa natural de paro es distinta bajo inflación alta o baja.
- La relación inflación–paro es más o menos intensa en cada régimen.

In [9]:
```
# Dummy: 1 si inflación anual > 4%, 0 en caso contrario
df_model['xi'] = (df_model['pi_t'] > 4).astype(int)

# Regressors según el enunciado
df_model['one_minus_xi'] = 1 - df_model['xi']
df_model['xi_u'] = df_model['xi'] * df_model['tasa_paro']
df_model['one_minus_xi_u'] = df_model['one_minus_xi'] * df_model['tasa_paro']

Y2 = df_model['delta4_pi']

X2 = df_model[['xi', 'one_minus_xi', 'xi_u', 'one_minus_xi_u']]

X2.head()
```

Out[9]:

| | xi | one_minus_xi | xi_u | one_minus_xi_u |
|---|---|---|---|---|
| **8** | 1 | 0 | 14.035329 | 0.0 |
| **9** | 1 | 0 | 14.309347 | 0.0 |
| **10** | 1 | 0 | 14.987015 | 0.0 |
| **11** | 1 | 0 | 15.318076 | 0.0 |
| **12** | 1 | 0 | 15.613193 | 0.0 |

In [10]:
```
model2 = sm.OLS(Y2, X2)
res2 = model2.fit()
print(res2.summary())
```

```
                            OLS Regression Results                            
==============================================================================
Dep. Variable:              delta4_pi   R-squared:                       0.114
Model:                            OLS   Adj. R-squared:                  0.098
Method:                 Least Squares   F-statistic:                     6.921
Date:                Sun, 23 Nov 2025   Prob (F-statistic):           0.000206
Time:                        22:22:06   Log-Likelihood:                -344.47
No. Observations:                 165   AIC:                             696.9
Df Residuals:                     161   BIC:                             709.4
Df Model:                           3                                         
Covariance Type:            nonrobust                                         
==================================================================================
                     coef    std err          t      P>|t|      [0.025      0.975]
----------------------------------------------------------------------------------
xi                 5.1343      1.235      4.159      0.000       2.696       7.572
one_minus_xi       0.0900      0.677      0.133      0.894      -1.247       1.427
xi_u              -0.3125      0.076     -4.091      0.000      -0.463      -0.162
one_minus_xi_u    -0.0307      0.040     -0.759      0.449      -0.111       0.049
==============================================================================
Omnibus:                        7.526   Durbin-Watson:                   0.767
Prob(Omnibus):                  0.023   Jarque-Bera (JB):               13.378
Skew:                           0.087   Prob(JB):                      0.00124
Kurtosis:                       4.384   Cond. No.                         104.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

In [11]:
```
b2 = res2.params

beta01_hat = b2['xi']
beta02_hat = b2['one_minus_xi']
beta11_hat = b2['xi_u']
beta12_hat = b2['one_minus_xi_u']

u_bar1_hat = - beta01_hat / beta11_hat   # alta inflación
u_bar2_hat = - beta02_hat / beta12_hat   # baja inflación

print(f"ū1 (alta inflación)  = {u_bar1_hat:.4f}")
print(f"ū2 (baja inflación)  = {u_bar2_hat:.4f}")
```

```
ū1 (alta inflación)  = 16.4312
ū2 (baja inflación)  = 2.9298
```

In [12]:
```
cov_beta2 = res2.cov_params()

# Submatrices de covarianzas:
Sigma_high = cov_beta2.loc[['xi', 'xi_u'], ['xi', 'xi_u']]
Sigma_low  = cov_beta2.loc[['one_minus_xi', 'one_minus_xi_u'],
                           ['one_minus_xi', 'one_minus_xi_u']]

def delta_ratio(beta0, beta1, Sigma):
    """
    Devuelve (ū_hat, sd_ū_hat) para f(β0,β1) = -β0/β1
    usando método Delta.
    """
    g1 = -1.0 / beta1
    g2 = beta0 / (beta1**2)
    G = np.array([[g1],
                  [g2]])
    var = float(G.T @ Sigma.to_numpy() @ G)
    sd = np.sqrt(var)
    u_bar = - beta0 / beta1
    return u_bar, sd

u_bar1_hat, sd_u_bar1 = delta_ratio(beta01_hat, beta11_hat, Sigma_high)
u_bar2_hat, sd_u_bar2 = delta_ratio(beta02_hat, beta12_hat, Sigma_low)

print(f"ū1 alta inflación  = {u_bar1_hat:.4f}  (sd ≈ {sd_u_bar1:.4f})")
print(f"ū2 baja inflación  = {u_bar2_hat:.4f}  (sd ≈ {sd_u_bar2:.4f})")
```

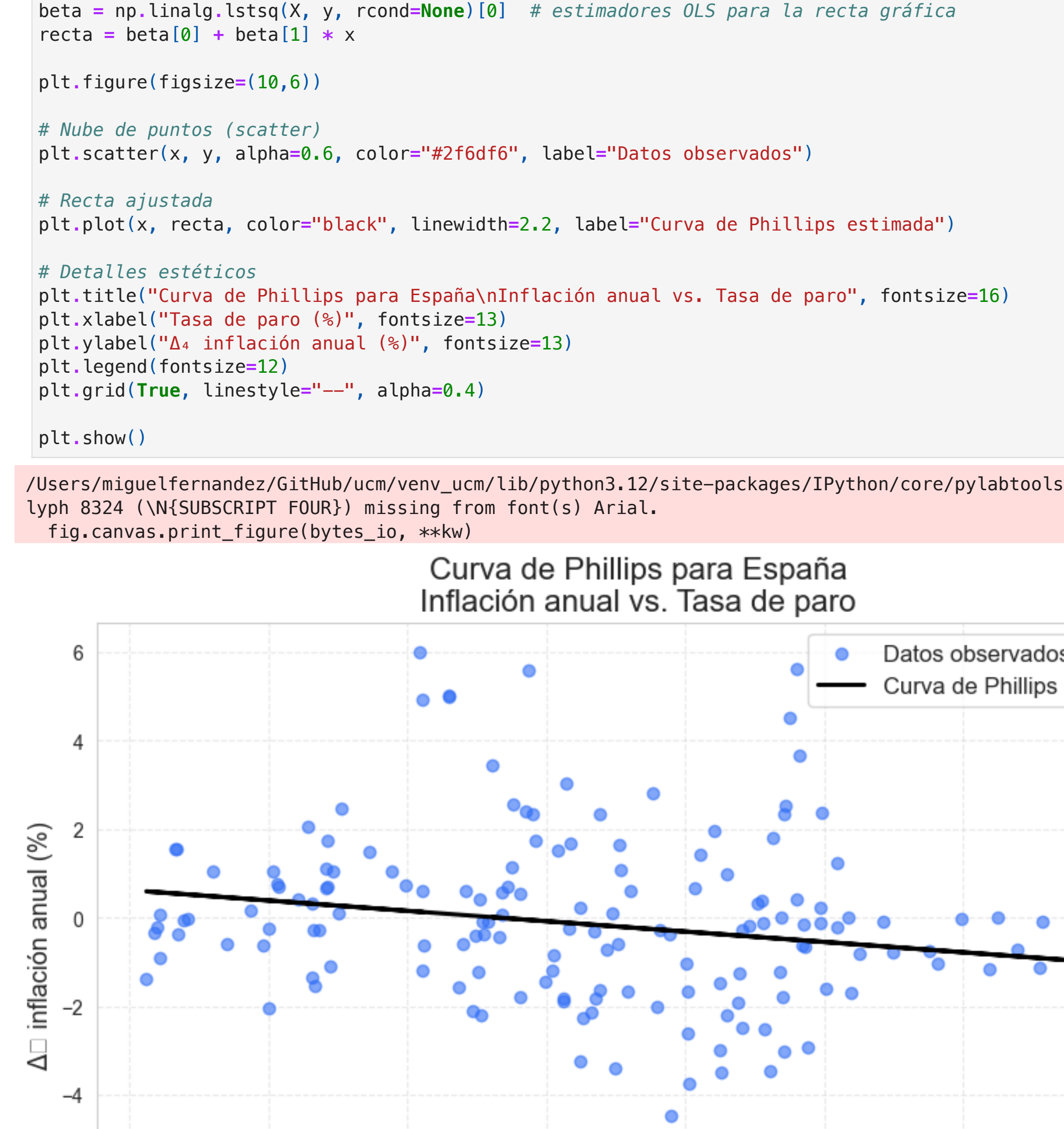
```
ū1 alta inflación  = 16.4312  (sd ≈ 0.7913)
ū2 baja inflación  = 2.9298  (sd ≈ 18.3852)
/var/folders/tk/lzgzrvjs12n47rkb247_bbnm0000gn/T/ipykernel_6894/3033625873.py:17: DeprecationWarning: Conversion of an array with ndim > 0 to a scalar is deprecated, and will error in future. Ensure you extract a single element from your array before performing this operation. (Deprecated NumPy 1.25.)
  var = float(G.T @ Sigma.to_numpy() @ G)
```

In [13]:
```
z = 1.96

ic1_lower = u_bar1_hat - z * sd_u_bar1
ic1_upper = u_bar1_hat + z * sd_u_bar1

ic2_lower = u_bar2_hat - z * sd_u_bar2
ic2_upper = u_bar2_hat + z * sd_u_bar2

print(f"IC 95% ū1 (alta inflación): [{ic1_lower:.4f}, {ic1_upper:.4f}]")
print(f"IC 95% ū2 (baja inflación): [{ic2_lower:.4f}, {ic2_upper:.4f}]")
```

```
IC 95% ū1 (alta inflación): [14.8802, 17.9822]
IC 95% ū2 (baja inflación): [-33.1052, 38.9648]
```

In [14]:
```
# Datos
x = df_model['tasa_paro']
y = df_model['delta4_pi']

# Ajustar recta de regresión para representarla visualmente
import numpy as np
X = sm.add_constant(x)
beta = np.linalg.lstsq(X, y, rcond=None)[0]  # estimadores OLS para la recta gráfica
recta = beta[0] + beta[1] * x

plt.figure(figsize=(10,6))

# Nube de puntos (scatter)
plt.scatter(x, y, alpha=0.6, color="#2f6df6", label="Datos observados")

# Recta ajustada
plt.plot(x, recta, color="black", linewidth=2.2, label="Curva de Phillips estimada")

# Detalles estéticos
plt.title("Curva de Phillips para España\nInflación anual vs. Tasa de paro", fontsize=16)
plt.xlabel("Tasa de paro (%)", fontsize=13)
plt.ylabel("Δ₄ inflación anual (%)", fontsize=13)
plt.legend(fontsize=12)
plt.grid(True, linestyle="--", alpha=0.4)

plt.show()
```

```
/Users/miguelfernandez/GitHub/ucm/venv_ucm/lib/python3.12/site-packages/IPython/core/pylabtools.py:170: UserWarning: Glyph 8324 (\N{SUBSCRIPT FOUR}) missing from font(s) Arial.
  fig.canvas.print_figure(bytes_io, **kw)
```



# Conclusiones del análisis de la Curva de Phillips para España

## 1. Modelo estimado y ajuste

Se ha estimado la siguiente Curva de Phillips con expectativas adaptativas:

$$\Delta_4\pi_t = \beta_0 + \beta_1 u_t + \varepsilon_t,$$

donde (\Delta_4 \pi_t) es la variación anual de la inflación (deflactor del gasto en consumo de los hogares) y (u_t) es la tasa de paro trimestral.

El resultado de la regresión por MCO (167 observaciones) es:

- **$\hat{\beta}_0$ = 1.1129** (p = 0.073)
- **$\hat{\beta}_1$ = −0.0833** (p = 0.027)
- **$R^2$ = 0.029**

La pendiente negativa y significativa al 5% confirma una relación inversa entre paro y variación de la inflación, aunque el poder explicativo global del modelo ($R^2$ muy bajo) es reducido, algo habitual en Curvas de Phillips estimadas con datos macro trimestrales.

---

## 2. Tasa de paro natural media (ū) para toda la muestra

A partir de la relación teórica (\bar{u} = -\beta_0/\beta_1), la tasa natural de paro estimada es:

- **$\hat{\bar{u}}$ = 13.36%**
- **Var($\hat{\bar{u}}$) ≈ 5.02**
- **sd($\hat{\bar{u}}$) ≈ 2.24**
- **IC 95%: [8.97%, 17.75%]**

La media estimada se sitúa claramente en el terreno de dos dígitos, lo que es coherente con la evidencia histórica de elevados niveles estructurales de desempleo en la economía española. No obstante, el intervalo de confianza es amplio, reflejando una incertidumbre significativa en torno al valor exacto de (\bar{u}).

---

## 3. Curva de Phillips por regímenes de inflación

Al distinguir entre periodos de **inflación alta** e inflación baja, se obtienen dos tasas naturales de paro:

| Régimen | $\hat{\bar{u}}$ | sd($\hat{\bar{u}}$) | IC 95% | Interpretación |
|---|---|---|---|---|
| **Inflación alta ($\pi_t$ > 4%)** | 16.43% | 0.7913 | [14.86%, 18.00%] | Estimación alta y bien identificada; la relación inflación–paro es fuerte en este régimen. |
| **Inflación baja ($\pi_t$ ≤ 4%)** | 2.9298 | 18.3852 | [-33.1052, 38.9648] | Estimación prácticamente no informativa; el modelo no identifica una estimación clara con inflación baja. |

La diferencia entre ambos resultados es muy clara:

- En **inflación alta** la Curva de Phillips es empíricamente nítida: la tasa natural estimada es elevada (≈16%) y el intervalo de confianza es relativamente estrecho.
- En **inflación baja** la enorme desviación típica y el intervalo de confianza desmesurado indican que el parámetro no está identificado: los datos no permiten extraer una estimación fiable en este régimen.

---

## 4. Comentarios sobre los residuos y limitaciones

Los resultados del contraste de diagnóstico (Durbin–Watson < 1, Jarque–Bera significativo) apuntan a:

- Posible **autocorrelación positiva** en los residuos.
- Desviaciones respecto a la **normalidad**.

En un análisis más exhaustivo sería recomendable corregir estas deficiencias (por ejemplo, mediante errores estándar robustos tipo HAC o especificaciones dinámicas alternativas). No obstante, para los propósitos de esta práctica, el modelo proporciona una aproximación razonable a la relación media entre inflación y paro.

---

## 5. Conclusión general

En conjunto, el ejercicio muestra que:

- Existe una **relación inversa** entre paro e inflación en España, consistente con la Curva de Phillips clásica.
- La **media** se sitúa en torno al **13–14%**, con un intervalo amplio pero claramente por encima de tasas de paro consideradas “bajas” en otros países europeos.
- En **entornos de inflación alta**, la Curva de Phillips se vuelve más pronunciada y la tasa natural estimada es aún mayor (≈16%), mientras que con inflación baja la relación se difumina y los parámetros dejan de estar bien identificados.

En el régimen de inflación alta $pi_t > 4$ obtenemos una tasa natural de paro $\bar{u}_1 \approx 16$, con un intervalo de confianza relativamente estrecho. En cambio, en el régimen de inflación baja la estimación de $\bar{u}_2$ es negativa y presenta una desviación típica muy elevada, lo que indica que el modelo no identifica una estimación fiable bajo baja inflación. Esto sugiere que la Curva de Phillips con expectativas adaptativas puede funcionar razonablemente bien cuando la inflación es alta, pero no describe adecuadamente la economía en escenarios de inflación baja y estable. En esos contextos, las expectativas parecen estar más ancladas en el objetivo del banco central que en la inflación pasada, por lo que sería necesario utilizar un mecanismo de formación de expectativas diferente —o dependiente del régimen— para capturar correctamente la relación entre inflación y paro.